A.1. Selección de colecciones

Para cada una de las siguientes necesidades, seleccionen una estructura apropiada del Java Collections
Framework.

Necesidades:
1. Mantener los paquetes pendientes conservando exactamente el orden en que fueron registrados.
2. Consultar rápidamente un paquete mediante su código.
3. Mantener el conjunto de municipios destino evitando duplicados.
4. Mantener los municipios ordenados alfabéticamente y sin elementos repetidos.
5. Atender paquetes teniendo en cuenta primero la mayor prioridad.
6. Mantener un historial de paquetes entregados respetando el orden de entrega.
7. Asociar cada código del paquete con el objeto Paquete correspondiente.

Para cada caso deberán completar la siguiente tabla:

Necesidad	Estructura elegida	Operación principal	Complejidad esperada	Alternativa descartada	Justificación
1. Mantener paquetes pendientes en orden exacto de llegada	LinkedHashMap< String, Paquete>	`put()`, iteración en orden	$O(1)$ promedio	ArrayList <Paquete>	**Justificación:** Mantiene una doble lista enlazada interna que preserva el orden de inserción estricto. Permite buscar/eliminar paquetes específicos en $O(1)$ sin recorrer toda la lista. **Sacrificio:** Consume más memoria RAM que un `ArrayList` al almacenar punteros extra (anterior/siguiente) por cada entrada
2. Consultar rápidamente un paquete por código	LinkedHashMap< String, Paquete>	get(código)	$O(1)$ promedio	ArrayList <Paquete>	**Justificación:** Aplica la función Hash sobre la clave código para acceder directamente a la posición de memoria en $O(1)$. **Sacrificio:** En el peor caso teórico de colisiones masivas degrada a $O(n)$, pero en Java $8+$ se auto-optimiza con árboles a $O(log\ n)$.
3. Mantener conjunto de municipios destino evitando	`TreeSet <String>` (`municipiosDestino`)	add (municipio)	$O(log\ n)$	HashSet <String>	**Justificación:** Un TreeSet garantiza unicidad mediante la comparación alfabética (compareTo) y evita

duplicados					elementos duplicados. **Sacrificio:** HashSet sería $O(1)$ para evitar duplicados, pero HashSet NO mantiene ningún orden. Se sacrificó un poco de velocidad ($O(log\ n)$ frente a $O(1)$) para cumplir con el orden alfabético automático sin hacer ordenamientos manuales extra
4. Mantener municipios ordenados alfabéticamente sin duplicados	`TreeSet <String>` (`municipiosDestino`)	Iteración / `iterator()`	$O(n)$	`ArrayList` + `Collections.sort()`	**Justificación:** Se basa en un Árbol Rojo-Negro (Red-Black Tree) balanceado. Los elementos ya están siempre ordenados de forma natural. Al iterar, los municipios salen en orden alfabético directo. **Sacrificio:** Inserción $O(log\ n)$ en lugar de $O(1)$.
5. Atender paquetes según mayor prioridad y reglas de empate	`PriorityQueue<Paquete>` (`colaPrioridad`)	`poll()` (extraer), `offer()` (insertar)	$O(log\ n)$	`LinkedList` o `ArrayList` ordenado	**Justificación:** Implementa un Montículo Binario (Heap). La extracción del elemento con mayor prioridad siempre toma $O(log\ n)$ al reestructurar el árbol, garantizando que la cabeza (peek) sea siempre el paquete más urgente según el Comparator multicriterio inyectado. **Sacrificio:** No permite acceso aleatorio por índice $O(1)$ Elementos intermedios.
6. Historial de paquetes entregados respetando orden	`HistorialEntregas<T>` (`Estructura Propia Genérica`)	`agregar(elemento)`	$O(n)$ en agregación al final (o $O(1)$ con puntero a cola)	`LinkedList` del JDK	**Justificación: Restricción estricta de negocio/cátedra. Se implementa una Lista Simplemente Enlazada Propia con nodos genéricos e interfaz `Iterable`/`Iterator` para**

					aislar punteros internos. **Sacrificio: obtener(posición) es $O(n)$ porque debe recorrer nodo por nodo desde la cabeza.**
7. Asociar cada código de paquete con el objeto Paquete correspondiente	`LinkedHashMap <String, Paquete> (paquetesPendientes)`	`get (codigo), containsKey(código)`	$O(1)$ promedio	`TreeMap <String, Paquete>`	**Justificación: Es una estructura clave-valor (Key-Value). Asociar codigo → Paquete permite validar unicidad en $O(1)$ mediante containsKey() antes de registrar un paquete nuevo.** **Sacrificio: Ocupa un espacio en memoria de $O(n)$ proporcional a las entradas guardadas**

A.2. Tiempo frente a memoria

Respondan:

Caso 1: La empresa actualmente maneja 30 paquetes, pero espera crecer hasta aproximadamente:

500.000 paquetes
La operación más frecuente será buscar un paquete por su código.
Comparen al menos:

- ArrayList<Paquete> y HashMap<String, Paquete>

1. ArrayList<Paquete>

- **Tiempo esperado de búsqueda:** $O(n)$ lineal. Para consultar 1 paquete en una lista de 500.000, en el peor de los casos (el paquete está al final o no existe) Java debe realizar hasta 500.000 **comparaciones con equals()**.
- **Memoria adicional:** Mínima ($O(n)$ bajo). Solo almacena un arreglo continuo de referencias/punteros a los objetos Paquete. Un arreglo de 500.000 elementos ocupa aproximadamente ~2 MB a ~4 MB solo en punteros.
- **Ventaja:** Muy eficiente en consumo de memoria RAM y excelente velocidad para iterar masivamente de forma secuencial gracias a la localización de caché del procesador.
- **Desventaja:** Extremadamente lento para búsquedas individuales por código. Si se hacen miles de consultas al día, el sistema colapsaría en tiempo de procesamiento.

2. HashMap<String, Paquete>

- **Tiempo esperado de búsqueda:** $O(1)$ constante (promedio). No importa si hay 30 o 500.000 paquetes; Java calcula la función Hash del código (ej. "PQ731".hashCode()), ubica el "bucket" en la tabla y recupera el paquete en 1 **o** 2 **pasos de memoria** (menos de 1 microsegundo).
- **Memoria adicional:** Significativa ($O(n)$ alto). Un HashMap requiere:
    1. Un arreglo de nodos (buckets) que suele dimensionarse con factor de carga (75%), creando espacio vacante para evitar colisiones.
    2. Nodos internos (Node<K,V>) que guardan el Hash de la clave, la Clave (String), el Valor (Paquete) y el puntero al siguiente nodo en caso de colisión. Para 500.000 paquetes, un HashMap puede consumir entre **30 MB y 50 MB de RAM adicional** solo en estructura.
- **Ventaja:** Búsquedas e inserciones instantáneas en $O(1)$ promedio.
- **Desventaja:** Alto consumo de memoria RAM y no mantiene ningún orden de inserción ni natural.

Estructura escogida: HashMap<String, Paquete> o LinkedHashMap para conservar el orden de llegada.

Justificación:

Aunque el HashMap requiere más memoria adicional (aproximadamente 40 MB extras para 500.000 registros, lo cual en servidores modernos es un consumo despreciable), **el costo crítico en un sistema de centro de distribución en tiempo real es el tiempo de respuesta**.

Realizar búsquedas $O(n)$ en un ArrayList requeriría ejecutar hasta 500.000 iteraciones por cada consulta de un cliente o funcionario, lo que degradaría el servidor. El HashMap garantiza respuestas en tiempo constante $O(1)$ de forma inmediata, logrando un sistema altamente escalable.

D.2. Una respuesta cuestionable de IA

Documenten como mínimo un caso donde una IA haya producido una solución:

● incorrecta;

● innecesariamente costosa;

● conceptualmente equivocada;

● que utilice una estructura inadecuada;

● que funcione pero tenga mala complejidad;

● que aplique búsqueda binaria sobre datos no ordenados;

● que confunda Iterable e Iterator;

● que sustituya la lista propia por LinkedList;

● o que no respete el alcance del curso.

Respuesta original	Problema detectado	Análisis	Solución final
Código propuesto por IA: colaPrioridad.remove(paqueteAsignar); al despachar un paquete por orden de llegada.	Complejidad ineficiente y mal uso de la estructura: Llamar a .remove(Object) sobre una PriorityQueue para eliminar un elemento intermedio.	• Restricción incumplida: El enunciado prohíbe explícitamente que el código cliente o externo acceda al primer nodo o a las referencias internas de la estructura para desplazarse. • Estructura utilizada: Exponer nodos (Nodo<T>) expone la arquitectura interna de la colección a fallos de punteros nulos (NullPointerException) o modificaciones no controladas desde el exterior .• Complejidad/Diseño: Forzaba a usar ciclos while(actual != null) fuera de la clase, rompiendo el contrato de la interfaz genérica ColeccionPropia<T>.	Solución Implementada: Se hizo la clase interna Nodo<T> estrictamente private static. Se implementó formalmente la interfaz Iterable<T> y la clase privada IteradorHistorial que implementa Iterator<T>. De esta forma, el historial se puede recorrer con un bucle for-each natural en el Main sin que el código externo conozca jamás la existencia de la clase Nodo ni maneje punteros manualmente.

Sugerir aplicar Búsqueda Binaria directamente sobre una lista devuelta por `obtenerPaquetesPorMunicipio()` o un `ArrayList` de paquetes sin realizar un ordenamiento previo explícito por código.	Violación de Precondición Algorítmica (Error Conceptualmente Equivocado): Ejecutar búsqueda binaria sobre datos no ordenados o asumir que un `ArrayList` mantiene orden natural automáticamente	• Comportamiento/Restricción: La Búsqueda Binaria ($O(log\ n)$) requiere de forma obligatoria e indispensable que la colección esté previamente ordenada bajo el criterio de búsqueda (código de paquete en orden ascendente). • Problema: Un ArrayList común preserva el orden de inserción o el resultado de un filtrado por municipio, no el orden alfabético natural. Si se ejecuta búsqueda binaria sobre una lista no ordenada, el algoritmo divide rangos basándose en comparaciones falsas, produciendo resultados incorrectos (falsos negativos) o bucles erróneos, rompiendo las precondiciones teóricas del algoritmo.	Solución Implementada: Se documentó que antes de invocar AnalisisDatos.buscarPaqueteBinario(), es obligatorio asegurar el orden de la colección. En la solución final se aplicó Collections.sort(lista) utilizando el orden natural (Comparable<Paquete> implementado en la clase) o se extrajeron los elementos ordenados desde una estructura tipo TreeSet antes de realizar la búsqueda $O(log\ n)$.
Crear un método `getHead()` o `getCabeza()` público que retorne el objeto `Nodo<T>` en la clase	Exponer las referencias de los nodos al código externo en lugar de abstraer el recorrido mediante `Iterable`	• Restricción incumplida: El enunciado prohíbe explícitamente que el código cliente o	Solución Implementada: Se hizo la clase interna Nodo<T>

`HistorialEntregas`, o permitir manipular el puntero `siguiente` directamente desde el `Main` para recorrer el historial.	e `Iterator`	externo acceda al primer nodo o a las referencias internas de la estructura para desplazarse. • Estructura utilizada: Exponer nodos (Nodo<T>) expone la arquitectura interna de la colección a fallos de punteros nulos (NullPointerException) o modificaciones no controladas desde el exterior. • Complejidad/Diseño: Forzaba a usar ciclos while(actual != null) fuera de la clase, rompiendo el contrato de la interfaz genérica ColeccionPropia<T> y el principio de ocultamiento de información de la POO	estrictamente private static. Se implementó formalmente la interfaz Iterable<T> y la clase privada IteradorHistorial que implementa Iterator<T>. De esta forma, el historial se puede recorrer con un bucle for-each natural en el Main sin que el código externo conozca jamás la existencia de la clase Nodo ni maneje punteros manualmente